

C03 — Structured Problem Solving: MECE, Issue Trees, Hypotheses

1. What interviewers test

The skill is turning an unclear question into parts that can be examined, ranked and answered in the time available, and then reasoning from a stated guess to a test to a conclusion. C02 trained you to reach a decision and show its cost. This chapter trains the step before it: finding out what you are deciding about.

Where it shows up. Inferred from the round structures candidates commonly report: the analytical or execution round ("a metric dropped, why?", "why did usage fall after launch?"), the strategy or business round ("should the company enter this market?"), the first minutes of an estimation prompt (decomposition is structure, and C04 builds on this chapter), and the user-segmentation step of any product design answer (C07). On the job the same skill appears in incident reviews, in a vague executive ask like "engagement feels off," in a metric dip you have to explain before Friday, and in backlog debates where nobody agrees what the problem is.

What is scored. Google publicly describes hiring on general cognitive ability, role-related knowledge, leadership and "Googleyness" Public [V: summaries citing Google's how-we-hire page; Google re:Work structured-interviewing guide], and a structured analytical round is the usual place the first is observed Inferred. Amazon publishes principles that include Dive Deep and Insist on the Highest Standards Public [V: Amazon's Leadership Principles page, amazon.jobs], which in practice rewards candidates who separate symptom from cause and check their reasoning Inferred. The scorer's note tends to read "clear structure, prioritized, reached a conclusion" or "listed possibilities, did not narrow" Inferred.

Five observable behaviors, which the rest of the chapter trains:

1. You clarify the question and the decision before you structure.
2. Your structure is [[mece]] along a cut that serves the decision.
3. You prioritize branches and say why.
4. You state hypotheses with the evidence that would reject them.
5. You finish with a conclusion or next action, and you separate a symptom from its cause.

The failure this prevents. Four patterns cost candidates and new PMs credibility: brainstorming a long list of "possible reasons" with no order, reciting a named framework instead of thinking ([framework-theater]), defined in C03.7), boiling the ocean (an exhaustive tree with no priority, so no answer), and anchoring on the first idea and then collecting evidence for it.

2. The concept from first principles

C03.1 Clarify and frame: turning a vague prompt into a question

"Engagement is down" is a statement, not a question. Before you can structure anything you need a question with a decision attached. Framing has three jobs.

Pin the phenomenon. What exactly changed? Which [[metric]], defined how, by how much, since when, and was the change sudden or gradual? A drop in "engagement" can mean daily users, sessions per user or time in app, and each points at different causes.

Pin the decision. What would we do differently depending on the answer? If every plausible answer leads to the same action, the analysis is decoration. If the VP will only fund a fix on the checkout page, a diagnosis that ends at "users are less engaged" does not help.

Pin the scope. Which users, platforms, regions and time window; which constraints on time and people; and what has already happened (a release, a campaign, an outage, a holiday).

Good clarifying questions fall into five kinds:

Kind	Example	Why it changes the structure
Definition	"How is a session defined?"	Different definitions imply different causes
Magnitude and timing	"How large, since when, sudden or gradual?"	Sudden points to an event; gradual to a trend
Scope	"All platforms or one? New users or existing?"	Narrows the tree immediately
Context	"Anything launched or changed in that window?"	The cheapest hypothesis is often the last release
Decision	"Who acts on this, and what can they change?"	Fixes which branches are worth examining

Ask three to five, in about two minutes, choosing the ones whose answers would change your structure. When the interviewer says "you decide," state the assumption aloud and continue: "I'll assume the drop is about 8% over two weeks on all platforms; tell me if that's wrong." The frame card (FW-C03-01) records the result.

Why it works: the structure you build depends on the question you ask. A tree built for the wrong question produces branches that are neither wrong nor useful.

When not to use the full routine. If the prompt is already specific and bounded, ask one confirming question and move on. An estimation prompt with clear terms, or a follow-up inside an answer you already framed, does not need a new round of clarification.

Counterexample. A candidate asks eight clarifying questions in a ten-minute segment, spends six minutes on them, and never structures. The interviewer sees thoroughness without progress. Clarification has diminishing returns: after the three or four questions that could change your structure, the next question is stalling.

Hidden risk. Clarifying questions can be used to ask the interviewer to do your thinking ("what do you think caused it?"). Ask for facts and scope, not for the answer.

C03.2 MECE: the principle, bad versus good, choosing the cut

MECE (mutually exclusive, collectively exhaustive) is a way of splitting a problem into parts that do not overlap and together cover the whole. [[mece]] works because each failure of it has a cost. Overlap means you count or discuss the same thing twice and cannot tell which branch explains the result. A gap means the real cause may sit in a branch you never opened.

A bad split and a good one:

Whole	Bad split	Why it fails	Good split
Users who did not sign up	Mobile, iOS, new users	Overlaps (iOS is mobile; new users use both) and has gaps (web, returning users)	Platform: iOS, Android, web
Revenue fell	Marketing problem, product problem, bad luck	Mixes functions with a vague residual	Volume, conversion, price per order
Reasons for churn	Price, quality, competitors, other	"Other" is likely the largest bucket; "competitors" overlaps price and quality	Involuntary (payment failed); voluntary because the need is gone; voluntary because they still have the need and use something else

The cut is a choice. The same whole can be split in many MECE ways, and the right one is the cut that serves the decision. The test is: "If the answer is branch A rather than branch B, would I do something different?" Revenue can be cut by customer segment, by channel, by product line, or as volume $\times$ price. If the decision is where to send the growth team, cut by channel. If the decision is whether to change the price, cut volume from price. A MECE cut that leads to the same action in every branch is correct and useless.

Mathematical decompositions are MECE by construction. Revenue = visitors $\times$ conversion $\times$ order value. Every change in revenue maps to a change in some factor, and the factors do not overlap. For quantitative problems this is often the best first cut, and it is the basis for the metric trees in C05.

Common splits for product problems:

- By time: before and after an event; trend versus step.
- By funnel step: each user leaves at one step.
- By user attribute: new and returning; platform; region; acquisition source.
- By cause type: internal and external; product, data, market.
- By input: volume, rate, value.
- By participant: supply and demand; buyer and seller; user and advertiser.

Perfect MECE is rarely achievable and need not be. Aim for MECE enough: no important overlaps, gaps acknowledged out loud ("this covers the main causes; I'm leaving out regulatory change as unlikely"), and any "other" bucket small. A rule of thumb is that an "other" holding more than about a fifth of the whole means a branch is missing.

When not to use it. At the start of ideation, forcing MECE can suppress the odd-but-important idea. List freely first, then cluster.

Counterexample. A team structures user pain points into four tidy categories before interviewing anyone, then finds the most severe problem (a trust issue after a billing mistake) fits none of them and is forced into "other." For divergent work such as generating ideas or collecting pain points, list first and structure second. Use MECE to organize what you know and to avoid gaps in a diagnosis, not to constrain discovery.

Hidden risk. A neat MECE tree can look more certain than it is. Its branches are a claim about the problem's structure, and the claim can be wrong, which is why the next step tests the structure against facts.

C03.3 Issue trees: building, pruning, prioritizing

An **issue tree** is a hierarchy that breaks a question into MECE branches, down to items that can be tested. [[issue-tree]] makes the structure visible to you and to the interviewer.

Three kinds, defined by the root question:

Kind	Root question	Example
Diagnostic	Why did X happen?	Why did weekly retention drop?
Solution	How can we achieve Y?	How can we raise activation by 10%?
Decision	Should we do Z?	Should we build a CRM integration?

Building the tree

1. Write the root question from the frame card.
2. Choose the cut that serves the decision (C03.2).

3. Split into three to five branches. Fewer than three means a missing branch unless the split is exhaustive by construction (new versus returning users, internal versus external causes); more than five means the cut is too fine for this level.
4. Run the MECE tests (FW-C03-02).
5. Split each branch the same way until the leaves are testable: each leaf names a check you could do with a specific data pull, interview, release log or experiment.
6. Stop at two or three levels in an interview. Depth beyond that belongs to the branches you decide to investigate.

Pruning. Cross out branches that facts you already have rule out. The user told you the drop is sudden, so slow-moving causes such as gradual competitor gains drop down the list.

Prioritizing. A tree with twenty leaves is not an answer. Score each branch, or each leaf, on three things:

- **Likelihood:** given what you know, how likely is it to explain the problem?
- **Impact if true:** how much of the problem would it explain, and how much would fixing it matter?
- **Cost to check:** how fast and cheap is the check?

Check first what is likely and cheap, then what is likely and expensive, and defer what is unlikely. The scoring is qualitative (low, medium, high) unless you have numbers. This is the [[pareto-principle]] applied to analysis: a few branches usually explain most of the problem. The scored issue tree is FW-C03-03.

Limits. A tree is only as good as its cut. If the first branches all lead nowhere, step back and try a different cut rather than splitting further. A tree that is MECE but built along the wrong dimension produces confident dead ends.

C03.4 Hypothesis-driven working: stating, testing, discarding

A **hypothesis** is a specific, testable statement about a cause or an effect that could turn out to be false. [[hypothesis]]-driven working means you form a guess early, say what would prove it wrong, and test the cheapest rejecting evidence first. It is faster than collecting data and waiting to see a pattern, because the hypothesis tells you which data to look at.

Anatomy of a useful hypothesis. A claim, a mechanism, a prediction and a rejection condition:

- Claim: "The drop is concentrated in Android users on version 5.2."
- Mechanism: "The release changed how login tokens refresh."
- Prediction: "Android sessions per user fell after the release date and iOS did not."
- Rejection: "If iOS fell by a similar amount, or Android fell before the release, I reject it."

A guess like "something is wrong with Android" cannot be rejected, so it cannot be tested.

Hold several at once. One hypothesis invites anchoring. State two or three competing ones, and choose the check that best separates them. After each check, update: say which hypothesis the result lowers and which it raises, in a sentence.

Discard out loud. "That rules out a data problem, because the dashboard and the raw logs agree." Interviewers listen for the discarding as much as for the hypothesis. Candidates who never discard are not testing.

Risks.

- *Confirmation bias.* You look for evidence for your favorite. The defense is to design the test that would reject it.
- *Searching many cuts until something looks significant.* This produces false findings; each extra cut raises the chance that noise looks like a cause (multiple comparisons, C12).
- *Premature hypotheses.* With no domain knowledge, a forced hypothesis is a guess. Explore descriptively first, then hypothesize.

The Build-Measure-Learn loop. The same logic applied to building: make the smallest thing that tests an assumption, measure it against a criterion set in advance, and decide what to do. It is FW-C03-05 and is extended for new products in C26.

When not to use it. When the test costs less than the debate, run the test: changing a label and watching a metric is cheaper than a meeting. And when you honestly have no basis for a hypothesis, a short descriptive pass ("let me look at the breakdown by platform and week") is the better first move.

Counterexample. A team is asked why a newly launched region shows low usage. They have no history, no benchmark and no prior launch to compare to. Spending an hour on hypotheses is guessing. A day of descriptive analysis (who signed up, where they stop, what they search for) produces the hypotheses worth testing. Hypothesis-driven working is a way to focus effort once there is something to focus on.

C03.5 Reusable structures for product problems

Most product problems have one of a few shapes, and a stock structure gives you a sensible first cut so that your attention goes to the content. The library in FW-C03-06 holds six: the *funnel* (each step of a user path), *supply and demand* (buyers, sellers and the match between them), *segments* (new and returning, platform, region), *time* (before and after, step or trend, seasonality), *ecosystem* (partners, competitors, regulators) and *inputs* (volume × rate × value). Choose by asking which cut best serves the decision, say the structure aloud, and expect to use two: one to split the problem and a second to examine the branch you picked. A stock structure is a starting point. It misleads when the problem does not have its shape, such as treating a policy change as a funnel problem, and it can pull you toward the questions the structure makes easy. C13 applies several of these structures to metric changes.

C03.8 Symptoms versus causes, and the root-cause methods that survive scrutiny

A **symptom** is an observable sign of a problem, and a **root cause** is the underlying condition that, if removed, stops the problem from recurring. `[[root-cause]]` separates the two.

"Conversion fell" is a symptom. "A payment-library update broke one checkout step on older Android versions" is a cause.

The symptom test: if I fix this, does the problem stop recurring, or only this instance? A fix that handles only this instance treats a symptom.

Three layers. Distinguish the proximate cause (the immediate trigger), the contributing causes (conditions that let it happen or made it worse), and the root cause (the condition whose removal prevents recurrence). Most real failures have more than one contributing cause `Inferred`, and hunting for a single root cause can mislead. Ask which conditions, if changed, would have prevented the failure.

"Human error" is rarely a root cause. If a person made an error, the productive question is what made that error easy to make and hard to catch.

The five whys. The **five whys** is a technique that asks "why" repeatedly, usually about five times, until it reaches a cause that can be acted on. `[[five-whys]]` is simple and useful for linear causes. Its weaknesses are that it follows one chain, it stops where the questioner stops, and it can end at a person. Use it with a second method for branching failures (FW-C03-08).

Mitigate, then diagnose. When harm is ongoing and the cause is unknown, the first job is to limit the harm with an action you can reverse and whose effects you understand, such as pausing a rollout with a `[[feature-flag]]`. The second is to find the cause. The mistake is acting on a guess that changes the system in ways you cannot predict. The case in CS-C03-02 shows what happens when a fix is attempted before the cause is understood `[V: SEC administrative order on Knight Capital]`.

When fixing a symptom is right. If the root cause is outside your control (a carrier's delays), or its removal costs more than the harm, treating the symptom is a legitimate choice. Say you are doing it and what you give up.

Counterexample. A support team keeps resetting a customer's account when it locks. Each reset fixes the instance, and the lock recurs weekly. The pattern is the signal: a recurring instance-level fix means the cause was never found. Conversely, a team that insists on finding the root cause of a one-time outage that cost nothing and cannot recur spends effort for no gain. Scale the depth of the search to the cost of recurrence.

3. Frameworks

Eight frameworks, set out below. FW-C03-01 to FW-C03-04 are the core sequence for an analytical answer; FW-C03-05 is the loop for building; FW-C03-06 is the library of reusable structures (index section C03.5); FW-C03-07 is the time-pressure skeleton (C03.6, detailed in section 4); FW-C03-08 covers root-cause methods (C03.8).

FW-C03-01 · The frame card · Original

- **Inputs:** a vague prompt; the interviewer's or stakeholder's answers to three to five clarifying questions.
- **Outputs:** a half-page statement of the decision, the question, definitions, scope, success criterion, constraints and assumptions.
- **Assumptions:** someone can answer clarifying questions, or you can state reasonable assumptions and flag them.
- **Tradeoffs:** time spent framing versus time spent analyzing; a frame that is too tight can exclude the real cause.
- **Failure modes:** asking many questions and never framing; framing the question around a favored solution; leaving the decision unstated so every branch looks equally useful.
- **Worked example:** "Ratings dropped" becomes: decision (what to fix first), question (why did the average rating fall from 4.6 to 4.3 over two months), definitions (rating is the post-delivery star score), scope (all cities, all orders), success (identify the top cause with enough confidence to act), constraints (a day of analyst time). See the Baseline, Strong and Standout answers in section 6.

Field	Prompt
Decision	What will someone do differently depending on the answer?
Question	The prompt restated as a specific question
Definitions	How each term is measured
Scope	Segments, time window, platforms, regions
Success	What would count as a good enough answer
Constraints	Time, data, people
Assumptions	What you are taking as true until told otherwise

FW-C03-02 · The MECE cut tests · Original

- **Inputs:** a proposed split of a problem into branches.
- **Outputs:** a corrected split plus a note on what is left out on purpose.
- **Assumptions:** the whole is well defined; the cut matters for the decision.
- **Tradeoffs:** rigor versus speed; perfect exclusivity is rarely achievable and can cost time that is better spent testing.

- **Failure modes:** passing the overlap test but not the decision test (a clean cut that leads to the same action in every branch); hiding a large unexamined area in "other"; siblings at different levels of abstraction.
- **Worked example:** "mobile, iOS, new users" fails the overlap test (iOS is mobile) and the gap test (web and returning users are missing); replaced with "iOS, Android, web," then a second tree by new versus returning users if the decision needs it.

Five tests, in order:

1. **Overlap test.** Can any item fit in two branches? If yes, redraw.
2. **Gap test.** Is there a plausible item that fits no branch? If yes, add a branch or declare the exclusion.
3. **Decision test.** Would different branches lead to different actions? If not, change the cut.
4. **Size test.** Is "other" more than about a fifth of the whole? If so, find the missing branch.
5. **Level test.** Are siblings at the same level of abstraction? ("Price" and "iOS 17 users" are not siblings.)

FW-C03-03 · The scored issue tree · Original

- **Inputs:** a root question, a chosen cut, and whatever facts you already have.
- **Outputs:** a tree two to three levels deep, each leaf scored for likelihood, impact if true and cost to check, and a short ordered list of what to check first.
- **Assumptions:** scoring can be rough (low, medium, high); some checks are cheap enough to do early.
- **Tradeoffs:** careful scoring takes time; early facts may change the scores.
- **Failure modes:** scoring that justifies the branch you already like; treating every leaf as equally important; checking the easy leaf because it is easy rather than because it is likely (the streetlight effect, section 9).
- **Worked example:** the onboarding funnel tree in Worked example 1 scores five step branches, finds one step holds a quarter of all installs lost, and checks it first.

Branch or leaf	Likelihood	Impact if true	Cost to check	Check order
(name)	L / M / H	L / M / H	L / M / H	1, 2, 3...

Rule for ordering: high likelihood and low cost first; high likelihood and high cost second; low likelihood and low cost third; low likelihood and high cost last, or drop. Medium-likelihood leaves sit between the second and third groups, ordered by impact.

FW-C03-04 · The hypothesis card · Original

- **Inputs:** a leaf from the scored tree, plus the context.
- **Outputs:** a card stating the claim, mechanism, predictions, the test, the rejection condition, the action if true and the cost to check.
- **Assumptions:** the claim is specific enough to be false; the test data exists or can be obtained.

- **Tradeoffs:** writing cards slows the first minutes; it speeds the rest by preventing vague checks.
- **Failure modes:** a claim that cannot fail; a test that could not tell the hypothesis from its rivals; an action field left blank.
- **Worked example:** "The drop is concentrated in Android on version 5.2" with predictions for iOS and for dates, a rejection rule, and the action (roll back the flag).

Field	Content
Claim	One specific sentence
Because	The mechanism
We would also expect	Predictions that would be true if the claim is true
Test	The check and the data
Reject if	The result that would rule it out
If true, we will	The action
Cost to check	Time and effort

FW-C03-05 · The Build-Measure-Learn loop · Established

- **Origin:** Eric Ries, *The Lean Startup* (2011).
- **Inputs:** a risky assumption (what must be true for the idea to work) and the smallest test of it.
- **Outputs:** a measured result against a criterion set in advance, and a decision to continue, change course or stop.
- **Assumptions:** the assumption can be tested with real users or data quickly; the team will act on the result.
- **Tradeoffs:** speed of learning versus polish of what is shown to users; a small test can mislead if it is unrepresentative.
- **Failure modes:** building without a stated criterion; measuring what is easy rather than what tests the assumption; changing the criterion after the result; a loop too slow to inform decisions.
- **Worked example:** to test whether a weekly digest email raises retention, build a manual version for 500 users, measure 30-day return against a pre-set threshold, and decide on automation only if the threshold is met.

Steps: (1) name the assumption that, if false, makes the idea fail; (2) design the smallest build that tests it; (3) write the success criterion and decision rule before you run it; (4) run and measure; (5) learn, and decide: continue, change, or stop. The speed of the loop often matters more than the polish of any one pass: a team that completes ten loops learns more than a team that completes two excellent ones, provided each loop tests a real assumption.

FW-C03-06 · The structure library · Original

- **Inputs:** a problem type and the decision it serves.

- **Outputs:** a recommended starting cut, with the first leaf checks and the situations where it misleads.
- **Assumptions:** problems repeat in shape; a standard structure is a starting point and may be combined or replaced.
- **Tradeoffs:** speed versus fit; a stock structure can pull you toward the questions it makes easy.
- **Failure modes:** using a favorite structure for every problem; combining structures that overlap; skipping the decision test.
- **Worked example:** for "marketplace conversion fell," the supply-and-demand structure separates a shortage of listings from a drop in buyer intent, and a time structure then checks whether the change was a step or a trend (Worked example 2).

Structure	Root split	Use when	Typical first checks	Misleads when
Funnel	Each step in a sequence	A user path leads to an outcome	Step conversion and drop-off by step	Users skip steps or re-enter
Supply and demand	Buyers, sellers, matching	A two-sided or capacity-limited system	Requests, available supply, match rate	A third party (regulator, platform) is the driver
Segments	New and returning; platform; region; source	You suspect the change is not uniform	The metric by segment	Segments overlap or mix shifts (see [[mix-shift]], C13)
Time	Before and after; trend and step; seasonality	The change has a date	The metric plotted against launches and events	The timeline is confounded by several simultaneous changes
Ecosystem	The product, its partners, competitors, regulators	An external party could be the cause	Competitor moves, partner changes, policy	You assume outside causes before checking your own
Inputs	Volume × rate × value	The metric is a product of factors	Which factor moved	The factors are not independent

Combining. Most real problems need two: one for the cut that serves the decision and one for a second pass on the branch you picked. Name each structure aloud and say why.

FW-C03-07 · The time-pressure skeleton · Original

- **Inputs:** a prompt, ten minutes (or the portion of a 45-minute round you have), and a pen.
- **Outputs:** a spoken or written answer with a frame, a structure, a priority and a conclusion.
- **Assumptions:** the interviewer will answer clarifying questions and may redirect.
- **Tradeoffs:** structure first costs the first minutes; it saves the middle and protects the end.

- **Failure modes:** diving deep into the first branch; never reaching a conclusion; ignoring the interviewer's redirection.
- **Worked example:** the AI-assistant case in Worked example 4, with timings.

Steps: **Clarify and frame, Structure, Prioritize, Test, Conclude.** Detailed with time budgets in section 4.

FW-C03-08 · The root-cause toolkit · Established

- **Origin:** the five whys from Taiichi Ohno's account of the Toyota Production System (English edition 1988); the fishbone (cause-and-effect) diagram from Kaoru Ishikawa's quality-control work in Japan, commonly dated to the 1960s (some sources place its first use earlier); fault tree analysis developed at Bell Telephone Laboratories in the early 1960s, by H. A. Watson, for a US Air Force study of the Minuteman launch control system [V: fault tree origin from an encyclopedia entry; Ishikawa dating varies by source; Ohno's account of the five whys is listed in the reading list].
- **Inputs:** a failure or recurring problem, and what is known about its timeline.
- **Outputs:** a causal explanation naming proximate cause, contributing causes and a root cause, with corrective actions.
- **Assumptions:** the failure can be traced; the team will look at process, not blame.
- **Tradeoffs:** depth versus time; the techniques are cheap and the discipline to apply them is the cost.
- **Failure modes:** five whys that stops at "human error"; a fishbone used as a brainstorm with no testing; a fault tree drawn from memory without evidence; multiple causes collapsed into one.
- **Worked example:** a notification outage analyzed by the five whys (a linear chain) and a fault tree (an AND of two conditions), shown below.

Which methods survive scrutiny in product work.

Method	Best for	Strength	Weakness	Verdict
Five whys	Linear causes in a process	Fast; builds the habit of asking why	One chain; stops where you stop	Keep, with discipline
Fishbone diagram	Brainstorming contributing causes by category	Prompts breadth	Lists possibilities without testing them	Use as a prompt, not an analysis
Fault tree	Failures that need several conditions together (AND) or any of several (OR)	Shows logical structure and single points of failure	Time-consuming; needs evidence	Keep for launch and incident work
MECE issue tree	Diagnosis before the cause is known	Ties to the decision	Depends on the cut	The default for interviews

A short example. A push-notification feature sent no notifications for six hours. Five whys: (1) Why did users receive none? The send job did not run. (2) Why? The scheduler's credentials had expired. (3) Why? The renewal reminder went to a former employee's mailbox. (4) Why?

Ownership of the credential was never reassigned. (5) Why? No checklist covers offboarding for service credentials. Cause: a process gap, not a person. A fault tree for the same outage shows an AND: the outage required both an expired credential and no monitoring alert on the send volume. Removing either condition would have prevented or shortened it, so the corrective actions are an ownership check and a send-volume alert.

4. Answer architecture

Three question types use this chapter directly. The skeleton in FW-C03-07 is the same for all three; what changes is the root question and the cut.

C03.6 Structuring under time pressure: the skeleton, the budget and the signposts

The skeleton: Clarify and frame, Structure, Prioritize, Test, Conclude.

1. **Clarify and frame** (about 20% of the time). Ask the three to five questions that could change your structure. State the frame in two sentences: the decision, the question, the scope and the assumptions.
2. **Structure** (about 20%). Name the cut and say why it serves the decision. Lay out three to five branches. Say what you are excluding on purpose.
3. **Prioritize** (about 15%). Pick the two or three branches you will examine first, and say why: likelihood, impact, cost to check.
4. **Test** (about 35%). For each chosen branch, state a hypothesis, the check, and what you would conclude. If the interviewer gives data, update aloud: which hypothesis it lowers and which it raises.
5. **Conclude** (about 10%). State what you believe, how confident you are, and the next action. If it is incomplete, say what you would do next and what would change your view.

Time budget for a ten-minute segment

Step	Minutes	What to produce
Clarify and frame	2	Three to five questions; a two-sentence frame
Structure	2	A named cut, three to five branches, stated exclusions
Prioritize	1.5	Two or three branches chosen, with reasons
Test	3.5	Hypothesis, check and conclusion per chosen branch
Conclude	1	Belief, confidence, next action

In a 45-minute round you may run this skeleton once at length or two to three times on short sub-problems, and the interviewer will steer. Treat their redirection as data, not interruption.

Signposts that keep you and the interviewer oriented

- "Let me take a moment to structure this." (Thirty to sixty seconds of silence is acceptable and better than rambling.)
- "I'll cut this by funnel step because the decision is which step to fix."
- "That gives four branches. I'll look at two first, because they are the most likely and the cheapest to check."
- "That rules out X because Y. That raises Z."
- "To summarize: I believe it's A; I'm moderately confident; the next step is B; I'd change my view if C."

Pacing rules. Go broad before deep: set the top-level branches before descending into any one. Spend depth on the branches you prioritized. Check in at natural breaks ("Shall I go deeper on supply or move to demand?") without asking permission for every step. If you are stuck, back up one level and ask what other cut would serve the decision.

Type A: the open-ended diagnostic ("Why did X happen?")

Root question: "Why did [metric] change?" Frame the metric, magnitude, timing and scope; choose a cut by what you can act on (funnel, supply and demand, segments or time, FW-C03-06); build a scored tree; test the top branches with hypothesis cards; conclude with the cause you believe, your confidence, and the action. C13 adds the product-metric diagnostic sequence in full detail, including data checks and a taxonomy of causes.

Type B: the decision question ("Should we do X?")

Root question: "Should we do X?" A decision tree splits into four questions that are exclusive in purpose and together cover most product decisions:

1. **Value:** is the need real and how big (who, how many, how painful, how much revenue or retention)?
2. **Feasibility:** can we build and run it (effort, dependencies, risk)?
3. **Viability:** does it work for the business (cost, pricing, strategic fit)?
4. **Risk and reversibility:** what could go wrong, and can we undo it (FW-C02-04)?

Evaluate each with evidence you have and evidence you lack, then recommend with the cost stated (FW-C02-03). The structure echoes the value, feasibility and viability lens familiar from design practice, with risk and reversibility added because C02 treats them as deciding variables.

Type C: "Structure this problem" with no data

Example prompt: "How would you think about improving engagement on a music app?" The skeleton is the same, with two differences: define the term first ("engagement" is not a metric until you choose one), and give the cut and the priority, not a conclusion. A good answer ends with a structured plan: the chosen metric, the branches, the two you would examine first, and what each check would tell you.

5. Worked examples

All four examples are **Illustrative**: the products are fictional and every number is invented. Each one is chosen to show a different structure from FW-C03-06 and a different decision.

Worked example 1 — Consumer subscription: why do first lessons go unfinished? (Illustrative)

Prompt. A language-learning app finds that only 38% of installs complete a first lesson (invented). "Why aren't people finishing?"

Clarify and frame. Questions asked: how is "completed first lesson" defined (a lesson reaching the final screen), how long has the figure been at this level (stable for three months), and does it differ by acquisition source. Answers (invented): ad-driven installs complete at 28% and organic installs at 51%, and 56% of installs are ad-driven. Check: $0.56 \times 28\% + 0.44 \times 51\% = 15.7\% + 22.4\% = 38.1\%$, which matches the blended figure, so the segment data and the headline agree.

Frame card: decision, which part of the first-session experience to fix first; question, where and why installs stop before finishing a lesson; scope, all new installs, last 90 days; success, the one or two steps that explain most of the loss; constraint, one week of analyst time.

Choose the cut. The decision is which step to fix, so a funnel cut serves it (FW-C03-06). Each install leaves at exactly one step, which makes the branches exclusive.

Build and score the tree. Root: "Why do 62% of installs not finish a first lesson?" Step conversion data (invented), with the product of the steps matching the headline:

Step	Conversion from previous step	Of 100 installs, still present	Lost at this step
Install to first open	90%	90.0	10.0
Open to account created	72%	64.8	25.2
Account to goal setup	85%	55.1	9.7
Setup to lesson started	80%	44.1	11.0
Lesson started to finished	86%	37.9	6.2

Check: $0.90 \times 0.72 \times 0.85 \times 0.80 \times 0.86 = 0.379$, which matches 38%. Of the 62 installs lost per 100, 25 are lost at account creation, the largest single loss.

Branch	Likelihood	Impact if true	Cost to check	Order
Account creation step	High (largest loss)	High (25 of 62)	Low (data in hand)	1
Source mix (ad versus organic)	High (gap of 23 points)	Medium	Low	2
Setup to lesson started	Medium	Medium (11 of 62)	Low	3
Install to first open	Medium	Medium (10 of 62)	Medium (needs store data)	4
Instrumentation error	Low	High	Low	check once, early

Hypotheses for branch 1. (H1) Forcing account creation before any value is shown loses casual users. (H2) Email verification adds friction. (H3) Permission prompts shown on the same screen cause abandonment. Tests: compare drop-off by sign-up method (social login versus email); measure time on the screen; watch ten session recordings. Rejection: if social-login users drop at the same rate as email users, H2 is rejected; if the drop is the same for installs that saw no permission prompt, H3 is rejected.

Second look at source mix. The ad-versus-organic gap says either the account step is worse for ad-driven users (intent mismatch) or they differ elsewhere. Splitting the account-step conversion by source separates them.

Conclude. "I believe the account step is the main cause: it loses a quarter of installs, and the gap between sources suggests ad-driven users arrive with less intent. My confidence is medium until I see the split by sign-up method. The reversible action is to test trying a first lesson before sign-up behind a [[feature-flag]] for 20% of new installs, with 30-day [[retention]] as a [[guardrail-metric]] so that we do not trade completion for later churn."

What this shows. The decision chose the cut (funnel). Arithmetic on the data checked that the tree was complete. Scoring ordered the work. Two structures were combined (funnel, then segments).

Worked example 2 — Marketplace: wait times doubled (Illustrative)

Prompt. In one city of a ride-hailing app, median rider wait time rose from 4 to 8 minutes over six weeks (invented). "What's going on?"

Clarify and frame. Definition: wait is from request to driver arrival. Gradual over six weeks, not a step. One city, not others. Nothing launched in the dispatch system in that window, according to the team (invented).

Choose the cut. A wait is an outcome of three things: how many riders ask (demand), how many drivers are available (supply), and how well the two are matched (matching). That is a supply-and-demand structure. Add a measurement branch because a changed definition would look identical.

Tested for MECE: demand, supply, matching and measurement together cover the ways the number can move. The draft "bad weather, drivers, app bugs" failed the tests: weather and bugs describe causes, drivers describes an actor, and the three do not share a level.

Priorities. The change was gradual, so a recent release is less likely and slow-moving supply or demand shifts are more likely. All four checks are cheap; run measurement first as a one-minute check, then the others in parallel.

Data from the interviewer (invented). Requests per day rose 5%. Online driver-hours fell 18%. So requests per online driver-hour rose by about 28% (1.05 divided by 0.82 is 1.28). Wait times in a capacity-limited system rise faster than the load does when utilization is high, so a 28% rise in load per driver can double waits. The magnitude is plausible, and the check is whether pickup distances rose as expected.

Next level under supply. Online driver-hours = active drivers × hours per active driver. Data (invented): active drivers fell 6% and hours per active driver fell 13%, and $0.94 \times 0.87 = 0.82$, matching the 18% fall. Most of the fall is in hours per driver, which is a behavior change, not attrition.

Hypotheses. (H1) Earnings per online hour fell, so drivers work less. (H2) A competitor raised incentives and drivers are splitting hours. (H3) A recent change to the app's incentive rules reduced guaranteed earnings. Checks: earnings per online hour by week; driver survey; incentive calendar. Data (invented): earnings per online hour fell 11% in week two, when an incentive rule was narrowed.

Conclude. "Supply of driver hours is the cause, and the likely trigger is the incentive change in week two. I'm fairly confident, because the timing, the size and the mechanism line up. Mitigation: reinstate targeted incentives in the affected hours, which is reversible, and track cost per ride as a [[counter-metric]]. Then confirm with a survey whether the competitor hypothesis also contributes."

What this shows. The cut came from the system's equation. Arithmetic on the data confirmed that the sub-branches add up. The decision was reversible and had a counter-metric.

Worked example 3 — B2B: a decision tree for "should we build the integration?" (Illustrative)

Prompt. A sales-analytics product is asked to build an integration with a popular customer-relationship system. "Should we?"

Frame. The decision is build now, build later, partner or decline. Success is a choice we can defend with evidence and a stated cost.

Decision tree. Root question: "Should we build it?" Four branches (Type B structure):

1. **Value.** Of the last 40 lost deals, 12 cited the missing integration (invented), worth about \$30,000 a year each, or \$360,000 in annual revenue (invented). Caution: lost-deal reasons are what buyers say, and the [[say-do-gap]] (C06) applies; check against deals won without the integration and against what support hears.
2. **Feasibility.** Twelve engineer-weeks to build and half an engineer ongoing (invented). The partner's programming interface is documented but rate-limited.
3. **Viability.** The partner's terms allow listing in their marketplace for a fee (invented). Dependency risk: the partner can change terms or compete.
4. **Risk and reversibility.** Building is partly reversible (the code can be retired, but customers who adopt it will expect it to remain). Classify with FW-C02-04: error cost medium, waiting cost moderate, reversibility partial, so stage it.

Options.

Option	Optimizes	Costs
Build the full integration	Win rate in lost-deal segment	Twelve engineer-weeks, ongoing load, dependency
Build a thin version for the top use case with five design partners	Evidence before commitment	Slower to market; partial coverage
Use a third-party connector tool	Speed	Margin and dependence on a third party
Decline	Engineering capacity	Continued losses in the segment

Recommendation. "I recommend the thin version with five design partners, time-boxed to six weeks. It gives up full coverage now to avoid committing twelve engineer-weeks on evidence that is partly self-reported. It is reversible in the sense that it can be retired before general launch. I would expand if at least three of the five partners use it weekly and the sales team shows a measurable change in the segment's win rate, and I would stop if partner terms change."

What this shows. A decision tree, unlike a diagnostic tree, asks whether to act. The branches are different questions (value, feasibility, viability, risk) rather than possible causes. The C02 tools (reversibility, cost of delay) supply the last branch.

Worked example 4 — Mixed: diagnose and decide in ten minutes (Illustrative)

Prompt (ten minutes). A software company's customer-support AI assistant shows the thumbs-down rate on answers rising from 8% to 12% over two weeks (invented). "What do you do?"

Minute 0 to 2: clarify and frame. *Asked:* what counts as a thumbs-down (the share of answers rated, down, over answers rated), how many answers are rated (about 5%, invented, so selection matters), when exactly did it start (it rose gradually then stepped up

around day three), and what changed recently. *Frame*: decision, whether to roll back a change or keep the system as is; question, why feedback worsened; scope, all topics, the last two weeks.

Minute 2 to 4: structure. Cut by what could have changed (the decision is what to roll back):

1. **Measurement**: the feedback widget, the share of people who rate, or the mix of who rates.
2. **Inputs**: the questions asked (a new product launch, a seasonal topic).
3. **System**: the assistant's model, instructions, retrieval index or knowledge base.
4. **Expectations and interface**: how answers are presented, or what users expect.

"I'm excluding the model provider's outage as separate; I'd treat it under system."

Minute 4 to 5.5: prioritize. The change was gradual then stepped, which suggests an event on day three. System changes in the last two weeks are the cheapest to check (a release log) and the most reversible. Question mix is cheap too (classify a sample). Measurement is a quick sanity check. I would check system and question mix first.

Minute 5.5 to 9: test. *H1*: a system change on day three degraded quality. Prediction: the thumbs-down rate rose across topics, not in one. *Reject if* the rise is confined to a topic with a recent product change. *H2*: the question mix shifted toward topics where the assistant is weak. Prediction: the rise is concentrated in new topics. *Reject if* the rate rose equally within unchanged topics. The interviewer provides data (invented): a knowledge-base re-index ran on day three; the rise appears across topics, including old ones; question mix is unchanged. "That lowers H2 and raises H1. It also fits the step on day three."

Minute 9 to 10: conclude. "I believe the re-index degraded retrieval, with medium-to-high confidence. First I would mitigate, because customers are getting worse answers now: switch to the previous index behind a flag, which is reversible and which I understand. Then I would diagnose why the new index is worse, by comparing retrieved passages for a sample of questions, and add a quality check to the re-index process. I would change my view if rolling back does not restore the rate." (The evaluation of AI products is developed in C24 and C25.)

What this shows. The skeleton fits ten minutes, with updates aloud as data arrives, and ends with mitigate-then-diagnose rather than a diagnosis alone.

6. Baseline, Strong, Standout

Prompt: "Customer ratings for our food-delivery app dropped from 4.6 to 4.3 over the last two months. How would you figure out what's going on?" (Numbers invented.)

Baseline answer

"There could be a lot of reasons: late deliveries, food quality, driver behavior, app bugs, or new competitors. I'd look at customer feedback to see what people complain about most, then work on fixing that. We should also talk to the operations team."

What is wrong: a list with no structure, no clarification, no priority and no test. "Look at feedback" is a task, not a hypothesis, and it ends in an action with no decision behind it.

Strong answer

"First I'd clarify: is the rating the post-delivery star score, has it dropped everywhere or in some cities, and has anything changed in the last two months? Assume it's all cities and gradual. The decision is where operations should focus, so I'll cut by stage of the order: ordering, restaurant preparation, delivery, and resolution after a problem. I'll check measurement too, in case the rating prompt changed.

"I'd start with delivery and preparation, because they're the most common drivers of low ratings and the data is available. I'd look at average delivery time and the share of late orders by city, and at ratings by restaurant type. If late orders rose in a few cities, that points to a courier-supply issue there. If ratings fell mainly for one restaurant category, preparation is the issue. I'd conclude by naming the segment that explains most of the drop and propose a fix for it."

Standout answer

Everything in the Strong answer, plus:

"Before choosing a cut, I'd ask what a drop of 0.3 implies, because the shape of the distribution changes where I look. If 11% of orders went from 4.6 to 2.0 (an assumed example), the average falls by about 0.3 while the other 89% are unchanged. So I'd first check whether the drop comes from a small group of bad orders or a general slide, by looking at the share of one- and two-star orders. That one check steers the whole investigation.

"I'd also test measurement and mix before operations, because they're cheap and would change the story: did the rating prompt change, did the share of customers who rate change, and did the mix of new customers shift? New customers often rate lower, so a surge in new users could lower the average without any change in service (a mix shift). The decision is where to act, so I'd then cut by order stage and look at concentrations.

"I'd write the first hypothesis so it can be rejected: 'Late deliveries in two cities explain most of the drop.' If late orders are flat, I reject it. If the fall is concentrated, I'd propose a mitigation that is reversible, such as courier incentives in those cities, while I look for the cause. I'd say how confident I am after each check and what would change my view."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Clarifies the metric, scope and timing	Reasons about what the size of the change implies
Builds a cut tied to a decision	Checks measurement and mix before operations
Prioritizes with reasons	Writes hypotheses with rejection conditions
Ends with a segment and a fix	Proposes a reversible mitigation and states confidence and what would change the view

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Google [as of 2026-10]. Google's four published hiring attributes are listed in C02. Reported APM loops include product, analytical and estimation rounds where visible structure is expected. Inferred. Practical consequence: say the cut and why you chose it, because the interviewer cannot score structure you keep in your head.

Company lens: Amazon [as of 2026-10]. Amazon publishes its Leadership Principles, which include Dive Deep and Insist on the Highest Standards. Public [V: Amazon's Leadership Principles page, amazon.jobs]. Candidates commonly report that follow-up questions test whether you have checked a number or a claim. Inferred. Practical consequence: after each hypothesis, say how you would verify it and what data you would pull.

Company lens: Meta [as of 2026-10]. Candidates commonly report analytical (execution) rounds with metric-change questions and follow-ups on what you would measure and what the metric could hide. Inferred [unverified: candidate-reported]. Practical consequence: check measurement and mix shift early; interviewers often provide data and expect you to update aloud.

Company lens: Microsoft [as of 2026-10]. Microsoft publicly emphasizes a growth-mindset culture. Public. Reports suggest interviewers appreciate a candidate who revises a hypothesis when data contradicts it. Inferred. Practical consequence: say "that rules out X" when it does, without defensiveness.

Company lens: Apple [as of 2026-10]. Third-party prep guides report team-specific loops and product-sense or technical deep-dive rounds, with structured reasoning expected inside product questions rather than as a separate drill Inferred [unverified: candidate-reported] . Practical consequence: use the skeleton inside a product answer (for example, structuring the cause of a quality complaint), and include craft and consistency among your branches.

Company lens: Netflix [as of 2026-10]. Netflix's published culture material describes giving people context instead of controls and expecting informed judgment Public [V: Netflix culture memo, June 2024] . Candidates commonly report prompts about content and subscription decisions in which a structure plus a way to test it is valued Inferred . Practical consequence: after you structure the problem, say which branch you would check first, with what data, and what you would decide on the result.

Company lens: Stripe [as of 2026-10]. Stripe is widely described as valuing clear written reasoning and precise thinking about developer and payments products Inferred [V: The Pragmatic Engineer, Inside Stripe's Engineering Culture Part 2 (strong culture of writing); Slab interview with Stripe's former Documentation Manager] . Candidates report case and product-sense prompts for payments and developer tools Inferred . Practical consequence: for an API or payments problem, structure by the developer funnel (sign up, integrate, go live, grow volume) and by failure type (declines, errors, latency), and write the structure down so it can be checked.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Starting to list causes before clarifying	Your first sentence is "it could be..."	Ask the three to five questions that change your structure
2	A tree with no decision behind it	You cannot say what you would do differently in branch A versus B	Apply the decision test (FW-C03-02)
3	Overlapping branches	An item fits in two places	Run the overlap test and redraw
4	A large "other"	"Other" is the biggest bucket	Find the missing branch
5	Depth-first on the first branch	Five minutes in, only one branch is examined	Set the top level first; prioritize
6	A hypothesis that cannot fail	No result would change your mind	Add a rejection condition
7	Never discarding	You collect evidence but never say what it rules out	After each check, say what it lowers and raises

#	Mistake	How to detect it	Fix
8	Checking the easy thing	You examine a branch because the data is handy	Rank by likelihood and impact, not only cost
9	Stopping at a symptom	Your "cause" is the metric restated	Ask whether fixing it stops recurrence
10	No conclusion	You end with "I'd look into these"	Close with belief, confidence and next action

C03.7 Failure modes: framework theater, overlapping branches, false completeness

Framework theater. [[framework-theater]] is naming or drawing a framework without using it to reach a conclusion. It looks like structure and is a substitute for it. Detect it by asking: did any step of the framework change what I said next? If the answer is no, the framework was decoration. *Examples:* reciting the four lettered steps of a memorized method with no link to the prompt; drawing a tree whose branches do not narrow anything; ending a framework with "so, it depends." *Fix:* use only the steps the question needs, say why you skip the others, and tie each step to the decision.

Overlapping branches. The mistake in C03.2: the same item sits in two branches, so evidence can be counted twice or credited to the wrong branch. *Detect:* try to place a concrete example in the tree; if it fits two places, the tree overlaps. *Fix:* redraw along a single dimension at each level.

False completeness. A tree that looks exhaustive but hides a gap, often in "other," in an unexamined assumption, or in a level where siblings are not comparable. It is dangerous because it signals confidence. *Detect:* ask "what real cause would not fit anywhere?" and "what percentage sits in other?" *Fix:* name what the tree excludes on purpose, and revisit the cut if the top branches yield nothing.

Two more that cost time: **anchoring** (holding the first hypothesis too long) and **the endless tree** (adding levels instead of testing). The defenses are competing hypotheses and a stopping rule: stop splitting when each leaf is a check you can run.

9. Mental models applied

Four models, each tied to a decision in this chapter. C02's models also apply here: [[first-principles-reasoning]] helps separate the facts of a problem from its conventions, [[inversion]] sharpens a pre-mortem on your own tree, and [[sunk-cost]] is the reason to abandon a tree that is not working.

The Pareto principle (MM-pareto) says that a small share of causes usually produces a large share of effects. The [[pareto-principle]] is why you prioritize: in Worked example 1, one of five steps held 25 of the 62 lost installs, and checking that first was most of the answer. It is a pattern, not a law: the split is often not 80 and 20, and in fraud or safety the long tail of rare events can matter more than the head.

Occam's razor (MM-occams-razor) says to prefer the simplest explanation consistent with the evidence. Application: before blaming a competitor or a market shift, check the last release and the data pipeline, which are simpler and cheaper to rule out. The limit is that complex systems often have several contributing causes, so "simple first" does not mean "simple only."

Chesterton's fence (MM-chestertons-fence) says not to remove a rule or practice until you know why it exists. Application: a team wants to cut an apparently pointless approval step in refunds. Investigate first; the step may be a fraud control. This is the same discipline as separating fixed facts from conventions (C02.2), applied to a process you are about to change.

The streetlight effect (MM-streetlight) is searching where it is easy to look rather than where the answer is likely to be. Application: analysts spend a week in the dashboard because that is where the data lives, while the cause sits in support tickets or an offline event the dashboard does not capture. Counter it by ranking branches by likelihood and impact before cost, and by asking what data you do not have.

Engineering, design and data-science perspectives

- **Engineering.** Engineers often think in components and dependencies, which makes them natural fault-tree thinkers. Ask for a dependency map when a failure spans systems, and ask what changed (deploys, configuration, flags) before asking what is broken. Blameless post-incident reviews, which look for contributing conditions rather than culprits, are the engineering form of C03.8.
- **Design and research.** Qualitative evidence belongs at the leaves of a tree: interviews and session recordings generate hypotheses that data then sizes, and the [[say-do-gap]] (C06) is a reason not to rely on either alone.
- **Data science.** Ask what a test could not detect. A hypothesis that survives a weak test has not been confirmed. Searching many cuts until one looks significant produces false causes, which is the multiple-comparisons problem in C12.

Checklist: a tree before you commit to it

1. Is the root a question with a decision attached?
2. Does the cut serve that decision (the decision test)?
3. Do the branches pass the overlap and gap tests?
4. Are there three to five branches per level, at the same level of abstraction?
5. Is "other" small, or have I named what I excluded?
6. Is every leaf a check I can run?
7. Have I scored branches and ordered the checks?
8. Does each hypothesis have a rejection condition?

10. Case studies

CS-C03-01 · The grocery checkout drop · Illustrative · Beginner

Fictional composite. Every number is invented.

Situation. An online grocer's checkout completion fell from 71% to 59% over three weeks (invented). The product team's first proposal is to redesign the checkout page.

Constraints. Two weeks, one analyst, and a redesign would take two months.

Data. Step conversions (invented), before and after:

Checkout step	Before	After
Cart to address entered	92%	92%
Address to delivery slot chosen	88%	74%
Slot chosen to payment	90%	90%
Payment to confirmation	97%	97%

Overall completion: $0.92 \times 0.88 \times 0.90 \times 0.97 = 70.7\%$ before and $0.92 \times 0.74 \times 0.90 \times 0.97 = 59.4\%$ after, matching the headline.

Problem definition. The loss is concentrated at one step, delivery-slot selection, not across checkout. A redesign of the whole page does not target it.

Analysis. Hypotheses for the slot step: (H1) slots became unavailable; (H2) the slot interface has a bug; (H3) delivery fees at that step rose. Checks: the share of sessions that saw "no slots available" rose from 6% to 21% (invented), which supports H1; error logs showed no change in slot-page errors (rejecting H2); delivery fees were unchanged (rejecting H3). Asking why slots were unavailable led to operations, which had cut delivery windows in two zones after a driver shortage (invented), and the product team had not been told. The symptom was low checkout completion; the cause was reduced delivery capacity combined with a slot step that showed an empty state with no alternative.

Options.

Option	Optimizes	Costs
A. Redesign checkout	Overall polish	Two months; does not address capacity
B. Restore capacity in the two zones	Directly removes the cause	Operations cost; depends on drivers
C. Show the next available slot and offer a waitlist when none exist	Recovers some lost orders now	Build effort of two weeks; does not add capacity

Tradeoffs. B and C address different parts of the problem (supply and experience). C is reversible and fast; B costs money and takes time, and the business must decide whether the orders recovered are worth it.

Decision. Ship C now, and bring the finding to operations with a number: each point of slot availability in those zones is worth a stated number of orders per week (invented). Drop the redesign.

Outcome. In this fictional scenario completion recovered to 66% within three weeks (invented), and operations restored part of the capacity after seeing the order-loss estimate.

Alternative outcomes. If the slot-page error rate had risen, H2 would have been confirmed and the fix would have been engineering's. If all steps had declined a little, the funnel cut would have given no concentration, and a segment cut (new versus returning, device) would have been the next try. The case shows the method: a cut that fits the decision, arithmetic that confirms the funnel, and hypotheses that can be rejected.

CS-C03-02 · Knight Capital, 2012: forty-five minutes, several causes · Verified · Advanced

Sources (types): the US Securities and Exchange Commission's press release of 16 October 2013 (2013-222) and its administrative order (Release No. 34-70694), both read in full, the order being the primary source for the findings; and Knight Capital Group's Form 8-K of 2013, for the company's own loss figure. The structure given to the analysis is the author's. Re-read in October 2026.

Situation. On August 1, 2012, a US trading firm, Knight Capital, deployed new software for a trading program connected to its automated order-routing system. During the first 45 minutes after the market opened, its router, SMARS, sent millions of child orders into the market while processing 212 small customer orders, producing more than 4 million executions in 154 stocks for more than 397 million shares, an unwanted net long position of about \$3.5 billion in 80 stocks and a net short position of about \$3.15 billion in 74 stocks, and a loss of more than \$460 million, according to the SEC's order (the SEC's press release describes the router sending "more than four million orders") [V: SEC order, paragraphs 1 and 17; SEC press release].

Constraints. The system was trading in real time in a live market; losses grew every minute; the cause was not known at the start; and the firm could not undo trades once executed.

Data. From the SEC release: the figures above; that an internal system generated 97 automated emails to a group of employees before the market opened, which identified an error but were not treated as alerts and were not acted on; and that the firm agreed to pay a \$12 million penalty without admitting or denying the findings. From the order itself [V: SEC order, paragraphs 13 to 27]: the new code was meant to replace unused Power Peg code and reused a flag that had once activated it; one of eight servers did not receive the new code because a technician did not copy it, no second technician reviewed the deployment, and there were no written procedures requiring one; on that server the flag woke the old code, which kept sending child orders because a function that counted filled shares had been moved in 2005 without the Power Peg code being retested; and during the incident Knight uninstalled the new code from the seven correct servers, which worsened the problem

because more orders then activated Power Peg. Knight's own filing reported a trading loss of \$457.6 million for the August 1 incident, and \$468.1 million including related legal and professional costs [V: Knight Form 8-K earnings exhibit, 2013].

Problem definition. At the time of the event, the symptom was unexpected, enormous order volume. The causal question was which component was sending orders and why. In hindsight, the problem was not a single bug but a set of conditions.

Analysis. *The author's structure.* A diagnostic tree for the first minutes might have split by where the orders originated: external (a customer, a market data fault) or internal (our router); then by which part of the router: new code, old code, or configuration; then by whether it was limited to some servers or all. The "one server out of eight" fact is what the tree would have isolated. In hindsight, a fault-tree view shows an AND of conditions that all had to be true: dead code still present on a production server, a flag that could activate it, a deployment that missed one server, a changed behavior in the old code that no test caught, and warnings that did not reach anyone as alarms. Remove any one and the event is smaller or does not happen. This is why searching for a single root cause misleads: the contributing causes are the story.

Options. *In the moment, as the author's reconstruction.* Stop the flow of orders from the router; roll back the new code; investigate before changing anything. Published accounts describe an attempt to remove the new code from the seven servers, which activated the old behavior on more of them and made the problem worse [V: SEC order, paragraph 27].

Tradeoffs. Speed against understanding. A reversible action whose effects are known (halting the router's order flow) differs from an action that changes the system's behavior on a guess (removing code). The first limits the harm; the second can multiply it.

Decision. As documented, the firm attempted the rollback, which the order says worsened the problem [V: SEC order, paragraph 27], and the loss accumulated until the problem was stopped.

Outcome. Documented: a loss of more than \$460 million in about 45 minutes and a \$12 million SEC penalty and an undertaking to retain an independent consultant to review its controls [V: SEC order]. The order records that Knight's parent combined with GETCO on 1 July 2013 to form KCG Holdings [V: SEC order, paragraph 11].

Alternative outcomes. *The author's analysis.* Verifying the deployment on every server before enabling the flag would have exposed the eighth server. Removing the dead code years earlier, or using a new flag instead of repurposing an old one, would have removed the trigger. Treating the pre-open emails as alerts would have given a chance to fix the problem before trading began. A staged release to one server first, with a check on order volume, would have shown the problem in seconds. These are the practices C17 develops ([staged-rollout], [feature-flag], [rollback]).

Lesson for an interview answer. Separate the symptom from the causes; expect several contributing causes; and when harm is ongoing and the cause is unknown, make the first action one you can reverse and understand. In a product interview, say it as: "I'd stop the harm first with something reversible, then find the cause."

11. Interview questions and model answers

Six questions, two per difficulty. Model answers are **Illustrative**; numbers are invented. Each is written against the five behaviors in section 1.

IQ-C03-01 · Beginner · structuring · PR

Prompt. We launched a "saved searches" feature and very few people use it. How would you find out why?

Rubric.

- **Baseline:** Lists reasons ("people don't know about it, it's not useful") and says "I'd survey users."
- **Strong:** Clarifies what "few" means, structures along an adoption path, prioritizes and states what each check would show.
- **Standout:** Separates "the need is small" from "the need is large but the feature is not found," writes rejectable hypotheses, and ties each result to a decision.

Model answer. First I'd clarify. What does "few" mean: a share of weekly users, and against what expectation? How long since launch, and on which platforms? The decision matters too: are we deciding whether to invest further, improve how people find it, or retire it?

I'll assume that 3% of weekly users have tried it against an expectation of 15% (invented), eight weeks after launch, and that the decision is whether to invest, fix discovery, or retire.

I'll structure along an adoption path, because the decision is which stage to fix: do people need this (is the underlying behavior common), do they see the entry point, do they try it, does it work when they try, and do they come back. Need comes first because it decides everything else: if almost nobody repeats searches, no discovery fix will help.

I'd check two things first, both cheap. Need: what share of users run the same search three or more times in a month? Say it is 25% (invented). Exposure: what share of those users ever saw the entry point? Say it is 20% (invented).

Hypotheses: H1, the need is real and the entry point is not seen. Prediction: among repeat searchers, few have seen the button, and those who did try it at a decent rate. I'd reject H1 if exposure is high and the try rate is still low. H2, the feature is seen but first use fails. Prediction: many starts, few saved searches. H3, the need is narrower than assumed. Prediction: few users repeat searches.

With the data I assumed, need is large and exposure is low, so H1 is supported and H3 is not. I'd test moving the entry point to the results page for 20% of users, with saved searches per active user as the metric and search satisfaction as a guardrail. I'd change my view if exposure rises and adoption stays flat, which would point to H2 and a usability problem.

Follow-ups.

- **Q.** What if exposure is high (most repeat searchers saw the button) and the try rate is low? **A.** "Then discovery is not the problem, and the hypothesis that people do not see it is rejected. I would look at the next steps on the path: the first-use experience and the

value after trying. I would ask five of those who saw it and did not try it why, and check what happens to the few who tried: do they save a second search and do they return to it? Low trial after high exposure suggests the label, the promise or the effort of setting it up is wrong, so I would test a different entry point or wording before concluding the need is weak."

- **Q.** How would you decide between investing further, fixing discovery and retiring it? **A.** "By the stage where the path breaks, which is why I structured it that way. If need is real and exposure is low, fix discovery. If need and exposure are fine but trial or return is low, improve the experience and retest. If the underlying behavior is rare (few users repeat searches), retire it and free the capacity. I would write the thresholds first, for example fix discovery if at least 25% of repeat searchers exist and under 30% have seen the entry (invented), so the choice follows the evidence and not my attachment."

IQ-C03-02 · Beginner · structuring · PR

Prompt. Should a ride-hailing app add a "schedule a ride in advance" feature?

Rubric.

- **Baseline:** "Yes, many apps have it" or a pros and cons list with no recommendation.
- **Strong:** Builds a decision structure (value, feasibility, viability, risk), evaluates each, and recommends with the cost stated.
- **Standout:** Identifies the core feasibility problem (promising availability in the future), proposes a staged test with a defined segment, and states what would change the answer.

Model answer. This is a decision question, so I'll use four branches that cover it: is the need real, can we deliver it, does it work for the business, and what is the risk and can we undo it.

Value: who needs to know a ride is arranged? Typically early-morning airport trips and people with fixed appointments. I'd check what share of rides today are requested on the same day for a known time, and how often riders cancel a ride they could not guarantee. Say 8% of rides fall in this group (invented).

Feasibility: the hard part is not the screen but the promise. At request time we cannot be sure a driver will be available in three days. Options include guaranteeing a driver by committing capacity, offering a best-effort reservation, or confirming a ride only an hour before. Each trades reliability for cost.

Viability: a guaranteed ride costs money when supply is thin, so we need a reservation fee or a cancellation rule, and drivers need a reason to accept scheduled rides.

Risk and reversibility: a failed scheduled ride is worse than a failed on-demand ride because a rider counted on it, and the damage to trust persists. But a pilot is reversible.

I recommend a pilot for airport and early-morning trips in one city, with a reservation fee, giving up broad coverage to learn whether we can keep the promise. The success metric is the on-time pickup rate for scheduled rides, with driver cancellation rate as the counter-metric. I would extend only if on-time pickup stays above the rate of on-demand rides. I would stop if the fee deters most riders or the promise fails often.

Follow-ups.

- **Q.** How would you decide the reservation fee and the cancellation rule for the pilot? **A.** "I would start from the cost of the promise. If guaranteeing a driver means paying a standby bonus on thin-supply mornings, the fee should cover the expected cost per scheduled ride, and the cancellation rule should protect the driver who kept the slot. I would test two fee levels in the pilot with a holdout and watch conversion, cancellations and driver acceptance of scheduled rides. The fee also signals seriousness: too low and riders reserve casually and cancel; too high and nobody reserves. I would choose the level where scheduled rides are fulfilled at the target rate at a margin that is not negative."
- **Q.** What would make you stop the pilot? **A.** "I would write kill criteria in advance: if fulfillment of scheduled rides is below 95% after four weeks, or if the contribution per scheduled ride stays negative after the fee, or if the cancellation rate by drivers is above a set level (invented thresholds), I would pause. A failed scheduled ride damages trust more than a failed on-demand ride, so the fulfillment criterion is the one I would not bend. A middle result, such as 96% fulfillment but negative margin, triggers a pricing change before expansion, not a launch."

IQ-C03-03 · Intermediate · structuring · PR

Prompt. On a marketplace for handmade goods, the number of listings is growing quickly but sales per listing are falling. Structure the problem.

Rubric.

- **Baseline:** Lists possible causes (competition, quality, pricing) without noticing that the metric is a ratio.
- **Strong:** Decomposes the ratio, separates supply from demand, and prioritizes.
- **Standout:** Notices the metric can fall mechanically when listings grow faster than demand, asks whether it is a problem, checks concentration, and links the finding to a decision.

Model answer. I'd start by noticing that "sales per listing" is a ratio: total sales divided by listings. It can fall because sales fall, because listings grow faster than sales, or both. So the first question is what happened to total sales. If total sales are flat while listings grow 30% (invented), sales per listing fall about 23% mechanically (1 divided by 1.3 is 0.77), and the cause is dilution, not a deterioration.

Then I'd ask whether it is a problem at all. For a marketplace, more supply is good if it serves unmet demand and bad if it spreads a fixed number of buyers more thinly and drives good sellers away. The decision is whether to limit or curate supply, improve discovery, or accept the change.

I'd structure with supply, demand and matching. Demand: buyers, conversion, repeat buying. Supply: listing quality, price, distribution across sellers. Matching: how well search and recommendations connect buyers to the listings that suit them.

Priorities: total sales trend (cheap, decisive), and the distribution of sales across listings (cheap): do the top 10% of listings take most sales, and is that share rising? A rising concentration says new listings are not being discovered or are weaker. Then check the quality of new listings and the match between new listings and search queries.

Hypotheses: H1, new listings are weaker and buyers favor established ones; prediction: new listings' conversion is lower and concentration rises. H2, demand is flat and supply growth is dilution; prediction: total sales flat, per-listing sales of old listings unchanged. I would reject H1 if new listings convert as well as old ones.

My recommendation depends on the result: if H2, focus on demand growth and avoid punishing sellers; if H1, work on discovery for new listings and on seller guidance.

Follow-ups.

- **Q.** Total sales are flat and the top 10% of listings take 80% of sales, up from 70%. What do you conclude? **A.** "That growth in listings is mostly dilution and discovery is concentrating demand on established listings: new listings are not being found or are weaker. I would not conclude yet that supply is too high. I would check quality (photos, descriptions, prices) of the new listings, how search ranks new items, and whether buyers who purchase from the long tail repeat. The decision then is between improving discovery for new listings, curating supply by quality, or accepting concentration if the buyer experience is good."
- **Q.** Would you ever limit the number of listings? **A.** "Possibly, but as a last resort. A cap hurts good new sellers and sends them elsewhere. I would first improve discovery and quality signals, and use lighter tools such as requiring minimum photo and description quality, or promoting new listings for a short period to give them a fair test. I would consider limiting supply in a specific category where oversupply clearly hurts buyers and sellers, measured by buyer conversion and seller churn, and I would test it in one category before wider use."

IQ-C03-04 · Intermediate · structuring · PR

Prompt. Your CEO says, "Users don't love our product." How would you turn that into something you can act on?

Rubric.

- **Baseline:** "I'd do a survey or NPS to see if that's true."
- **Strong:** Asks what the CEO observed, defines "love" as measurable behaviors, segments, and frames a question with a decision.

- **Standout:** Treats the statement as a hypothesis about a segment, converts it into several measurable claims, orders the checks, and agrees on the decision with the CEO before analysis.

Model answer. I would treat this as a hypothesis from someone with a signal, and my first job is to learn what the signal was. I'd ask the CEO what prompted it: a number, a review, a conversation, or a competitor comparison. I'd also ask what decision it should drive: a roadmap change, a retention program, a repositioning.

Then I'd turn "love" into behaviors we can measure: users return without prompting (retention and frequency), they recommend (referrals, ratings), they would be unhappy to lose it (a standard survey question such as how disappointed they would be), and they pay or upgrade. I'd also ask: which users? "Users" is rarely uniform; the statement may be true for one segment and false for another.

So the frame becomes: "Which user segments retain and recommend at lower rates than we expect, and what do they say is missing?" The decision: which segment and which problem the next quarter's roadmap should address.

Structure by cohort and segment: new versus returning, by use case, by acquisition source. Prioritize: retention by cohort and segment is already available, so I'd start there. Then qualitative checks on the weakest segment: ten interviews and a read of the support themes.

I'd come back to the CEO within a week with a one-page answer: which segments are weaker, how much, and what the weakest segment says. If the data shows retention is healthy, I'd say so, and ask what else prompted the concern, because the signal may be about something I haven't measured, such as word of mouth.

Follow-ups.

- **Q.** The CEO says it was a competitor's product that made them feel this. How do you proceed? **A.** "I would turn the comparison into specific questions: what does the competitor do better, for which users and tasks, and what do our users who also use it say? I would check whether our retention differs among users who use both products, and read reviews that mention the competitor. I would not copy their feature list; I would find the job where they win and test whether it matters for our segment. Then I would report back with the segment, the problem and one test, so the CEO sees the signal turned into a decision."
- **Q.** How do you bring the answer back to the CEO in a week? **A.** "With a one-page answer first: the segment that retains and recommends at lower rates than we expect, the evidence (cohort retention and the survey), what interviews say is missing, and the proposed decision for next quarter's roadmap. I would state my confidence and what I could not check. I would also say what would change my conclusion. If the signal turned out to be one conversation and the data show no problem, I would say that too and thank them for prompting the check."

IQ-C03-05 · Hard · structuring · PR

Prompt. Churn at a B2B product has doubled in two quarters. You have ten minutes. Structure the problem and tell me the first three things you'd check and why.

Rubric.

- **Baseline:** Lists causes (price, competitors, product, service) and says "look at customer feedback."
- **Strong:** Clarifies churn's definition and timing, builds a MECE structure, chooses three checks with reasons.
- **Standout:** Checks measurement and renewal timing before causes, uses concentration to decide where to look, orders checks by likelihood and cost, and includes a reversible mitigation.

Model answer. *Clarify (two minutes).* Is churn counted by customers or by revenue? Is it logos lost or contracts not renewed? Doubling from what, say from 2% to 4% a quarter (invented)? Was it gradual or a step? Is it concentrated in a segment, a plan or a size? Did anything change: price, packaging, support, product? And what is the decision: where to focus a retention effort?

Structure. Churn is a ratio too: customers lost divided by customers up for renewal. So I'd ask first whether the numerator or the denominator changed, which means checking renewal timing: a large annual cohort renewing this quarter would raise churn without any change in behavior. Then I'd cut by who left (segment and size) and why (cause type: product value, price, competition, service, customer's own situation).

First three checks, with reasons.

1. **Measurement and renewal mix.** Cheap, and it could change the whole story. Compare churn within each renewal cohort rather than in aggregate. If churn within cohorts is flat, the doubling is a timing effect.
2. **Concentration.** Cheap and high impact. Which segment, plan or size explains most of the lost revenue? If three large accounts explain half of it, the response is different from a broad slide.
3. **Recent changes.** The release, pricing and support log for the two quarters. A price rise or a plan change would show as churn concentrated in the affected plan after its date.

Hypotheses. H1: the change is a renewal-mix effect; I'd reject it if churn within each cohort also rose. H2: a pricing or packaging change drove churn in one plan; I'd reject it if the plan's churn rate did not change after the date. H3: competitor pressure in one segment; I'd look for churned accounts that moved to the same alternative.

Conclude. "I'd report which of these holds with a confidence level. In parallel I'd start a reversible mitigation for at-risk accounts in the affected segment: a customer-success call and an offer review for accounts that are close to renewal. I'd change course if the check shows churn is a mix effect, in which case the mitigation is unnecessary and I would put the effort into confirming the forecast for the next quarter."

Follow-ups.

- **Q.** Churn within each renewal cohort is flat. What do you conclude, and what next? **A.** "That the doubling is a timing effect: a large cohort came up for renewal this quarter, so the aggregate rose without a change in behavior. I would tell leadership that the retention of each cohort is stable, show the renewal calendar for the next four quarters so they can see when large cohorts return, and still check concentration and reasons, since a stable rate can hide a quiet problem in a segment. I would also check that the measure counts customers or revenue consistently, because the conclusion differs if big accounts are driving it."
- **Q.** Three large accounts explain half of the lost revenue. What do you do? **A.** "The response becomes account-specific, not a broad program. I would find out why each left: a champion who departed, a price or contract issue, a missing capability, a competitor. I would look for early signals we missed, such as falling usage in the months before, and build an account-health view to catch them. I would also check whether other large accounts show the same warning signs and prioritize outreach to them, with a named owner for each and a review date."

IQ-C03-06 · Hard · behavioral · BE

Prompt. Tell me about a time your first diagnosis of a problem was wrong.

Rubric.

- **Baseline:** A story in which the candidate was "mostly right" or the error was somebody else's, with no change in behavior.
- **Strong:** States the first hypothesis, how it was tested, what showed it was wrong, and what the real cause was.
- **Standout:** Shows what the candidate did to avoid anchoring, how quickly they dropped the hypothesis, what they changed in their working method afterward, and the cost of the error.

Model answer. *Illustrative: a composite story; numbers invented.* "I was the PM for an online store's mobile checkout. Conversion fell from 3.1% to 2.6% over two weeks. My first hypothesis was that a new payment provider we had added caused failures, because the timing matched the release. I was fairly sure, and I told my manager that I thought it was payments.

"I checked it as a hypothesis I could reject. If payments were failing, payment success rates would have fallen. They had not: success was 97% before and after. I dropped the hypothesis the same day, about six hours after I had stated it.

"The next check was a cut by traffic source. The drop sat almost entirely in mobile web visitors from a new advertising campaign, which accounted for 30% of traffic in the second week and converted at 0.8% (invented). The overall rate fell because the mix changed, not because checkout worsened. Within each source, conversion was flat.

"The cost of my first diagnosis was about half a day of an engineer's time, which I had asked to review payment logs. What I changed afterward was my default order of checks. I now check who the traffic was before I check what the product did, and the first question in our weekly metric review is 'what changed in the audience?' I also write my hypothesis down with what would reject it before I look at data, because I noticed I had been looking for confirmation."

Follow-ups.

- **Q.** How did you avoid anchoring on your first hypothesis? **A.** "By turning it into a prediction I could reject. I wrote what payment failure would look like (success rates falling) and checked that first, which took minutes. Stating the hypothesis aloud to my manager felt risky, but it made me accountable for checking it. I also asked what else changed in the same two weeks, which opened the traffic-mix question. The habit I keep is to write the rejection criterion before looking at the data."
- **Q.** What did the wrong diagnosis cost, and how did you manage it with your team? **A.** "About half a day of one engineer's time reviewing payment logs, which I told the engineer honestly: I had been wrong and the log review was still useful because it ruled out a cause. I said so in the weekly metrics review without defensiveness, and I changed the default order of checks for everyone: traffic mix first, then product changes. Owning the miss early made it easier for others to say when they had been wrong."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Numbers are invented.

EX-C03-01 • MECE or not

Problem. For each split, say whether it is MECE, state the failure, and give a fix: (a) customers: enterprise, small business, mobile users; (b) revenue: visits $\times$ conversion rate $\times$ average order value; (c) causes of low sales pipeline: product, marketing, sales, other; (d) monthly users: new, returning, churned; (e) devices: iOS, Android, tablets, web.

Context. You are building trees for a B2B and consumer portfolio.

Data. None beyond the lists.

Constraints. One sentence per item.

Expected approach. Apply the overlap, gap, size and level tests (FW-C03-02).

Solution. (a) Not MECE: "mobile users" is a device, not a customer size, and overlaps both others. Fix: enterprise, mid-market, small business. (b) MECE by construction; every change in revenue is a change in one factor. (c) Not MECE: the first three are functions, "other" is likely large, and causes are not functions. Fix: split by funnel stage (awareness, interest, qualification, close) or by cause type (demand, targeting, execution). (d) MECE if status is defined at a date (new: first active this month; returning: active before and now; churned: active before and not now); note that churned users are not "monthly users" in the strict

sense, so define the whole accurately. (e) Not MECE: tablets overlap iOS and Android, and form factor is a different dimension from operating system. Fix: choose one dimension (operating system or form factor).

Alternative solution. Test each with a concrete example: "an iPad user": which branch? If two, it overlaps.

Tradeoffs. Fixing every overlap costs time; fix those that would put evidence in the wrong branch.

EX-C03-02 · Choose the cut

Problem. A subscription app's revenue fell 6% last month. For each decision, choose the MECE cut that serves it: (a) where the growth team should focus; (b) whether to change the price; (c) whether to change the plan structure.

Context. You can use any standard cut.

Data. Revenue = active subscribers $\times$ average revenue per subscriber (invented structure).

Constraints. Say why each cut serves the decision.

Expected approach. For each decision, ask: if the answer were branch A versus B, what would I do?

Solution. (a) Cut by acquisition channel: the action differs by channel (which to invest in). (b) Cut revenue as subscribers $\times$ average revenue per subscriber: if subscribers fell but price per subscriber held, a price change is the wrong lever; if average revenue per subscriber fell, price or mix matters. (c) Cut by plan mix: if subscribers moved to a cheaper plan, the structure is the issue. Each cut is MECE and leads to a different action.

Alternative solution. Two-level: first subscribers $\times$ revenue per subscriber, then channel within subscribers and plan mix within revenue per subscriber.

Tradeoffs. The two-level tree is more complete and slower; use it when time allows.

EX-C03-03 · Build and score a tree

Problem. Fewer new users complete profile setup. Build a scored issue tree and order the checks.

Context. Setup has four steps.

Data. *Invented.* Of 100 users who start: 85 finish name and photo, 60 finish interests, 52 connect contacts, 50 finish.

Constraints. Two levels deep; one page.

Expected approach. Choose a funnel cut, compute losses, score, check arithmetic.

Solution. Losses per 100 starts: name and photo 15, interests 25, contacts 8, final step 2; total 50. Interests holds half of all losses. Branches: interests step (high likelihood, high impact, low cost to check, order 1); name and photo (medium, medium, low, 2); contacts (low, low, low, 3); final (low, low, low, 4); instrumentation (low, high, low, check once).

Hypotheses for interests: too many required selections; unclear value; poor mobile layout. Checks: time on screen, required-selection count versus drop, session recordings. The loss figures sum to 50 and completion is 50%, matching the data.

Alternative solution. Add a segment cut (new platform versus others) if the funnel cut shows no concentration.

Tradeoffs. A funnel cut hides audience differences; add a segment look after the first check.

EX-C03-04 · Write hypotheses that can fail

Problem. Rewrite each into a hypothesis card with claim, because, we would also expect, reject if: (a) "Retention is bad." (b) "Users don't like the new design." (c) "Notifications are annoying."

Context. You are a PM with event and survey data.

Data. None specified.

Constraints. Each claim must name a segment or a mechanism.

Expected approach. Use FW-C03-04 and make each claim falsifiable.

Solution. (a) Claim: "Week-four retention of users who did not complete setup is at least 15 points lower than for those who did." Because: setup gives first value. Also expect: the gap is largest in the first two weeks. Reject if the gap is under 5 points. (b) Claim: "Task completion on the settings screen fell after the redesign." Because: controls were moved. Also expect: more taps to reach the same setting. Reject if completion and taps are unchanged. (c) Claim: "Users who receive more than three notifications a day turn them off at twice the rate of those who receive one." Because: frequency causes fatigue. Also expect: opt-out concentrated in high-frequency cohorts. Reject if opt-out rates are similar across frequency.

Alternative solution. Add a cost to check and an action if true to each card.

Tradeoffs. Specific claims may be too narrow; run several competing claims.

EX-C03-05 · Prune and prioritize

Problem. Order these leaves for checking: A (likelihood high, impact high, cost high), B (high, medium, low), C (low, high, low), D (medium, high, medium), E (low, low, high), F (high, high, low).

Context. A diagnostic tree for a drop in repeat purchases.

Data. As given.

Constraints. Apply the ordering rule in FW-C03-03 and name what you would drop.

Expected approach. High likelihood and low cost first; high likelihood and high cost second; low likelihood and low cost third; low likelihood and high cost last.

Solution. Order: F (high likelihood, high impact, low cost), B (high, medium, low), A (high, high, high cost), D (medium) before C? The rule puts low likelihood below medium; D (medium, high, medium cost) is next, then C (low likelihood but low cost, so a quick check), and E last or dropped (low likelihood, low impact, high cost). So: F, B, A, D, C, then drop E. Note that A is expensive and high-impact, so start it early in parallel if resources allow.

Alternative solution. Order by expected value ($\text{likelihood} \times \text{impact} \div \text{cost}$) with numbers 1 to 3, and compare with the rule.

Tradeoffs. Numeric scoring invites false precision; use it only to break ties.

EX-C03-06 · Five whys and a fault tree

Problem. A release shipped with a broken password-reset email for two days. Apply the five whys and a fault tree.

Context. Facts: the reset link used a hard-coded web address; staging used a different address, so tests passed; the release checklist had no reset test on production-like settings; monitoring did not track reset completion; the release went out on a Friday.

Data. As listed.

Constraints. Name corrective actions that do not blame a person.

Expected approach. Run a five-whys chain and then a fault tree with AND and OR.

Solution. Five whys: (1) Why did users not receive working links? The link pointed to the wrong address. (2) Why? The address was hard-coded. (3) Why was it not caught? Staging used a different address, so tests passed. (4) Why was it not caught later? No test on production-like settings, and no monitoring of reset completion. (5) Why? The release checklist had no such step. Fault tree: outage requires the defect AND no test catching it AND no monitor catching it; the Friday timing is a contributing factor in the duration. Actions: remove hard-coded addresses; add a production-like test; add a monitor on reset completion; avoid unmonitored Friday releases for authentication changes.

Alternative solution. Add a fishbone for contributing causes by category (process, tooling, people, timing).

Tradeoffs. More actions reduce risk and add process; choose those that cut the most cause at the least cost.

EX-C03-07 · Symptom or cause

Problem. Classify each statement as a symptom, a proximate cause or a root cause, and explain: (a) "Conversion fell." (b) "The new checkout button is hidden on small phones." (c) "The design review did not test small screens." (d) "Support tickets doubled." (e) "A caching change returned stale prices." (f) "There is no process to compare caching changes against price data."

Context. Two incidents are mixed together.

Data. As listed.

Constraints. One sentence each.

Expected approach. Apply the symptom test: does fixing it stop recurrence?

Solution. (a) Symptom. (b) Proximate cause of the conversion fall. (c) Contributing or root cause (the process gap that let it ship). (d) Symptom. (e) Proximate cause of stale prices and the ticket rise. (f) Root cause (missing control). Fixing (b) corrects the instance; fixing (c) and (f) prevents recurrence.

Alternative solution. Draw the two chains separately and mark where they would branch.

Tradeoffs. A deeper root cause is not always worth fixing; weigh the cost of recurrence.

EX-C03-08 · Structure a decision

Problem. Should a note-taking app localize into a new language?

Context. Engineering says localization takes eight weeks (invented).

Data. *Invented.* 14% of signups come from the language's region, with 40% lower activation than average; search shows rising interest; a competitor has localized.

Constraints. Use a Type B decision structure and make a recommendation with the cost stated.

Expected approach. Value, feasibility, viability, risk and reversibility.

Solution. Value: 14% of signups with low activation suggests latent demand blocked by language (hypothesis, not proof). Feasibility: eight weeks of engineering plus ongoing translation. Viability: depends on willingness to pay in the region; check pricing. Risk: low quality translation can harm trust; it is mostly reversible but a localized product creates expectations. Options: full localization; localize the first-run experience only; do nothing. Recommendation: localize onboarding and core screens first (about three weeks, invented), giving up full coverage to test whether activation rises. Expand if activation in the region improves by a set amount. Change the view if activation does not move.

Alternative solution. Run a fake-door test showing a "coming soon in your language" prompt and count clicks.

Tradeoffs. A partial localization is faster and may feel unfinished to users.

EX-C03-09 · Three-minute structuring drill

Problem. In three minutes, structure: "Why would users stop using a note-taking app after a month?"

Context. A timed spoken drill.

Data. None.

Constraints. Frame, structure, prioritize in 180 seconds.

Expected approach. Frame the metric; choose a lifecycle cut; name two branches to check first.

Solution. Frame (20 seconds): the question is why monthly retention falls after week four, and the decision is which stage to fix; assume it is lower than for similar products. Structure (60 seconds) by the user lifecycle: never reached value (setup and first notes), reached value but no habit (no trigger to return), switching or migration (found another tool, or sync issues), and changed circumstances (project ended). Prioritize (40 seconds): check what share of lapsed users created more than five notes; if most did not, the problem is early value; if they did, the problem is habit or switching. Say what you would do in each case. Conclude (60 seconds): name the first check and the reversible action.

Alternative solution. Cut by segment (students, professionals) first if the product has distinct audiences.

Tradeoffs. Three minutes allows breadth, not depth; say what you would do with more time.

EX-C03-10 · Spot false completeness

Problem. A team's tree for "why did app ratings fall?" has four branches: bugs, slow performance, competitors, other. In the data, 45% of low ratings fall under "other." Critique and fix the tree.

Context. The team was proud of the tree.

Data. Reasons were coded from 1,000 one-star reviews (invented): bugs 20%, performance 15%, competitors 5%, other 45%, price 15%.

Constraints. Show the revised tree and say what the first check is.

Expected approach. Apply the size test and the level test.

Solution. "Other" is 45% and price (15%) is hiding inside it, so the tree has false completeness. Bugs and performance are product quality; competitors is an external factor; they are not at the same level. Revise by cause type: product quality (bugs, performance), value and price, service and support, expectations set by marketing, external factors (competitors). Re-code "other" into these, then look at what remains. First check: which of the new categories grew in the period (ratings are a change over time, not a snapshot).

Alternative solution. Cluster the "other" reviews by topic with a simple keyword pass and look at the top five themes.

Tradeoffs. Re-coding takes time; a sample of 100 "other" reviews is usually enough to reveal missing branches.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: structure only.* Pick a prompt (use the exercises and IQ-C03-01 to IQ-C03-06). Set a three-minute timer. Frame, choose a cut, give three to five branches, and name the two you would check first. Stop at three minutes.

- *Week 2: the skeleton.* Use the ten-minute budget in section 4 on a diagnostic prompt, and speak aloud or record.
- *Week 3: add data.* Ask a partner to provide a data point at minute five and to redirect you once. Practice updating aloud.

After every drill (three minutes). Redraw your tree cleanly on one page and run the MECE cut tests on it. The redraw is where most of the learning happens.

Feedback checklist (yes, partly, no).

1. Did I clarify before structuring, and did I ask no more than five questions?
2. Did I say the decision behind the question?
3. Did I name the cut and why it serves the decision?
4. Do my branches pass the overlap and gap tests?
5. Is "other" small, or did I name what I excluded?
6. Did I prioritize and give a reason?
7. Did I state at least one hypothesis with a rejection condition?
8. Did I say what a result rules out?
9. Did I separate the symptom from the cause?
10. Did I end with a belief, a confidence level and a next action?

With a partner. Ask your partner to interrupt once with "what would that rule out?" and once with a new fact that contradicts your leading hypothesis. Note whether you update or defend.

Answer journal. For each drill record the date, the prompt, the cut you chose, your score out of ten, the item you will fix, and the cut you would try next time. After six drills, check which cuts you reach for most. If it is always the same one, practice a different structure from the library (FW-C03-06).

14. Implementation playbook

Apply the chapter in the next two weeks on real work. Each deliverable doubles as an interview story.

Days 1 to 2. Choose a live ambiguity: a metric dip, a recurring complaint, or a vague request. Write a frame card (FW-C03-01) and show it to the person who raised it; revise until they agree it captures the question and the decision.

Days 3 to 5. Build a scored issue tree (FW-C03-03). Run the MECE cut tests. Ask a colleague to place three real examples in the tree and tell you where they do not fit.

Week 2. Write hypothesis cards (FW-C03-04) for your top two branches, run the cheapest checks, and record what each rules out. Write a one-page memo using the decision-memo template from C02 (FW-C02-06) with the finding, the confidence, the action and what would change it.

Week 2. Run a five-whys, and then a fault tree, on one recent incident or miss from your team. Propose corrective actions that change a process, not a person.

Deliverables that become stories: the frame card and tree (a time you brought structure to ambiguity), the memo (a decision with a stated confidence), and the incident analysis (a time you found the cause behind a symptom). Keep the numbers real.

15. Key takeaways and further reading

Key takeaways

1. Clarify and frame before you structure: the decision behind the question decides the cut.
2. A MECE cut is only useful if the branches lead to different actions; choose the cut that serves the decision.
3. Prioritize branches by likelihood, impact and cost to check, and state hypotheses that can be rejected.
4. Separate symptoms from causes, expect several contributing causes, and when harm is ongoing, mitigate with something reversible before you diagnose.
5. Use frameworks for what the question needs; reciting one without using it is theater.

Further reading

- Barbara Minto, *The Pyramid Principle* (multiple editions). The source of MECE-style structured thinking and of the practice of leading with the answer.
- George Pólya, *How to Solve It* (1945). A classic account of problem-solving heuristics, including restating the problem and working backward.
- Eric Ries, *The Lean Startup* (2011). The Build-Measure-Learn loop used in FW-C03-05.
- Taiichi Ohno, *Toyota Production System: Beyond Large-Scale Production* (English edition 1988). The origin of the five whys.
- Karl Popper, *The Logic of Scientific Discovery* (English edition 1959). The case for hypotheses that can be falsified.
- Donella Meadows, *Thinking in Systems* (2008). Useful for the multi-cause, feedback-driven failures that single-cause analysis misses.
- Ethan Rasiel, *The McKinsey Way* (1999). A short, readable account of structured problem solving in consulting.